



Representación del conocimiento y Razonamiento

ASP-clingo

1. HECHOS

Átomos ground/hechos base

```
nombre(término*).
```

```
término = constante
```

```
*: 0 o más términos
```

```
lejano.
```

```
estudiante(luis).
```

```
amistad(luis,marta).
```

```
tiene_deuda(sara).
```

1. HECHOS

Átomos ground /hechos base negados (NEGACIÓN FUERTE)

```
- nombre(término*).
```

término = constante

*: 0 o más términos

```
- amistad(luis,marta).  
- tiene_deuda(sara).
```

2. REGLAS

1. Reglas lógicas de inferencia

cabeza :- cuerpo

<code>a :- b,...,m.</code>	<code>a,b,...,m</code> de cabeza y cuerpo son átomos del tipo nombre (término*), donde término = constante o variable
<code>a :- b_p,...,m_p, not b_n,...,not m_n.</code>	
<code>a :- b_p,...,m_p, -b_n,..., -m_n.</code>	

```
humano(X) :- estudiante(X).  
aprobado(X) :- estudiante(X), nota_media(X).  
beca(X) :- estudiante(X), nota_alta(X), renta_baja(X).
```

```
estudiante(eva).  
tiene_carnet(eva).  
  
puede_examinar(X) :- estudiante(X), tiene_carnet(X), not sancionado(X).
```

Negación por defecto: no se puede derivar que **eva** esté sancionada, es decir, no se puede derivar **sancionado(X)**. Por tanto, se satisface **not sancionado(X)** para X=eva, y se deriva que eva se puede examinar.

Modelos resultantes

- tiene_carnet(eva) estudiante(eva) puede_examinar(eva)

```
estudiante(eva).  
tiene_carnet(eva).
```

```
puede_examinar(X) :- estudiante(X), tiene_carnet(X), - sancionado(X).
```

Negación fuerte: no sabemos explícitamente si **eva** está sancionada, no se puede derivar `sancionado(X)`. Por tanto `- sancionado(X)` no se satisface y no se deriva que `eva` se pueda examinar.

```
estudiante(eva).  
tiene_carnet(eva).  
- sancionado(eva).  
  
puede_examinar(X) :- estudiante(X), tiene_carnet(X), - sancionado(X).
```

Negación fuerte: En este caso, sí sabemos explícitamente que **eva** no está sancionada, así que sí se deriva el átomo `puede_examinar(eva)`.

Modelos resultantes

- `-sancionado(eva) tiene_carnet(eva) estudiante(eva)`
`puede_examinar(eva)`

2. Restricciones de integridad

`:- cuerpo.`

No tienen cabeza y por tanto no derivan ningún átomo. El cuerpo representa una condición que no puede ser verdadera en ningún modelo estable: si el cuerpo se satisface, el modelo se descarta.

```
matriculado(eva,rcr).  
matriculado(eva,apa).  
horario(rcr,jueves).  
horario(apa,jueves).  
  
:- matriculado(X,A1), matriculado(X,A2), A1!=A2,  
   horario(A1,H), horario(A2,H).
```



no puede ocurrir en ningún modelo estable

2		1	
	4		3
3			
4		3	1

```
% Por sistema de coordenadas (X,Y)
```

```
sudoku(1,1,4).
```

```
sudoku(3,1,3).
```

```
sudoku(4,1,1).
```

```
sudoku(1,2,3).
```

```
sudoku(2,3,4).
```

```
sudoku(4,3,3).
```

```
sudoku(1,4,2).
```

```
sudoku(3,4,1).
```

```
% Restricción de columna
```

```
:- sudoku(X,Y1,V), sudoku(X,Y2,V), Y1 != Y2.
```

3. Reglas de elección

cabeza [:- cuerpo].

Las reglas de elección o *choice rules* siempre tienen una cabeza y el cuerpo es opcional. Si la regla no tiene cuerpo entonces la cabeza se aplica globalmente.

La cabeza de una regla de elección se define con la construcción { . . }.

{a}

a: átomo del tipo `nombre(término*)`, donde `término` = constante o variable

a es el átomo candidato que el solver ASP puede derivar/elegir como verdadero por la regla de elección

```
persona(eva).  
persona(maria).  
proyecto(p1).  
proyecto(p2).
```

```
{asignado(X,Y)} :- persona(X), proyecto(Y).
```

Para cada par (X,Y) tal que $\text{persona}(X)$ y $\text{proyecto}(Y)$ son ciertos, el átomo $\text{asignado}(X,Y)$ es una **opción**; es decir, el solver ASP decidirá en cada modelo estable si $\text{asignado}(X,Y)$ es verdadero o falso, respetando el resto de reglas y restricciones del programa.

Modelos resultantes

- proyecto(p1) proyecto(p2) persona(eva) persona(maria)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(maria,p1)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(maria,p1)
asignado(eva,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(maria,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p2)
asignado(maria,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(maria,p1)
asignado(maria,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(maria,p1)
asignado(eva,p2) asignado(maria,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
asignado(maria,p1)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
asignado(maria,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
asignado(maria,p1) asignado(maria,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
asignado(eva,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
asignado(eva,p2) asignado(maria,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
asignado(maria,p1) asignado(eva,p2)
- proyecto(p1) proyecto(p2) persona(eva) persona(maria) asignado(eva,p1)
asignado(maria,p1) asignado(eva,p2) asignado(maria,p2)

$\{a_1; a_2; \dots; a_n\}$

a_i : átomo del tipo nombre (término*), donde término_i = constante o variable

a_i son los átomos candidatos que el solver ASP puede elegir como verdadero por la regla de elección

```
persona(eva).  
persona(maria).
```

```
{jugadores(X,Y); jueces(X,Y)} :- persona(X), persona(Y), X != Y.
```

- nada: ni jugadores(X,Y) ni jueces(X,Y)
- solo jugadores(X,Y)
- solo jueces(X,Y)
- ambos: jugadores(X,Y) y jueces(X,Y) a la vez.

Modelos resultantes

- `persona(eva) persona(maria)`
- `persona(eva) persona(maria) jueces(maria,eva)`
- `persona(eva) persona(maria) jugadores(eva,maria)`
- `persona(eva) persona(maria) jugadores(eva,maria) jueces(maria,eva)`
- `persona(eva) persona(maria) jueces(eva,maria)`
- `persona(eva) persona(maria) jugadores(eva,maria) jueces(eva,maria)`
- `persona(eva) persona(maria) jueces(maria,eva) jueces(eva,maria)`
- `persona(eva) persona(maria) jugadores(eva,maria) jueces(maria,eva)`
`jueces(eva,maria)`
- `persona(eva) persona(maria) jugadores(maria,eva)`
- `persona(eva) persona(maria) jugadores(maria,eva) jueces(maria,eva)`
- `persona(eva) persona(maria) jugadores(maria,eva) jueces(eva,maria)`
- `persona(eva) persona(maria) jugadores(maria,eva) jueces(maria,eva)`
`jueces(eva,maria)`
- `persona(eva) persona(maria) jugadores(maria,eva) jugadores(eva,maria)`
- `persona(eva) persona(maria) jugadores(maria,eva) jugadores(eva,maria)`
`jueces(eva,maria)`
- `persona(eva) persona(maria) jugadores(maria,eva) jugadores(eva,maria)`
`jueces(maria,eva)`
- `persona(eva) persona(maria) jugadores(maria,eva) jugadores(eva,maria)`
`jueces(maria,eva) jueces(eva,maria)`

También es posible escribir una regla de elección sin cuerpo. Por ejemplo:

```
estudiante(eva).
```

```
{asignatura(eva,rcr); asignatura(eva,apa); asignatura(eva,eda)}.
```

Modelos estables resultantes:

- estudiante(eva)
- asignatura(eva,apa) estudiante(eva)
- asignatura(eva,eda) estudiante(eva)
- asignatura(eva,apa) asignatura(eva,eda) estudiante(eva)
- asignatura(eva,rcr) estudiante(eva)
- asignatura(eva,rcr) asignatura(eva,eda) estudiante(eva)
- asignatura(eva,rcr) asignatura(eva,apa) estudiante(eva)
- asignatura(eva,rcr) asignatura(eva,apa) asignatura(eva,eda)
 estudiante(eva)

$\{a_1: l_{11}, \dots, l_{1k}; \dots; a_n: l_{n1}, \dots, l_{nm}\}$

a_i : átomo del tipo `nombre(término*)`, donde `término` = constante o variable

l_{ij} : átomos del tipo `nombre(término*)`, donde `términoi` = constante o variables

$a_1, \dots, a_n$: átomos candidatos de elección

$l_{11}, \dots, l_{1k}; \dots; l_{n1}, \dots, l_{nm}$: condiciones locales para cada elección

```
persona(eva).  
persona(maria).  
imparcial(eva).  
imparcial(maria).
```

```
{jugadores(X,Y):compatible(X,Y);jueces(X,Y):imparcial(X),imparcial(Y)}  
:- persona(X), persona(Y), X != Y.
```

```
persona(eva).
persona(maria).
imparcial(eva).
imparcial(maria).
```

```
{jugadores(X,Y): compatible(X,Y);
 jueces(X,Y):imparcial(X),imparcial(Y)} :- persona(X), persona(Y), X != Y.
```

- imparcial(eva) imparcial(maria) persona(eva) persona(maria)
- imparcial(eva) imparcial(maria) jueces(eva,maria) persona(eva)
persona(maria)
- imparcial(eva) imparcial(maria) jueces(maria,eva) persona(eva)
persona(maria)
- imparcial(eva) imparcial(maria) jueces(eva,maria) jueces(maria,eva)
persona(eva) persona(maria)

```
persona(eva).  
persona(maria).  
imparcial(eva).  
imparcial(maria).
```

```
{jugadores(X,Y): compatible(X,Y), not jugadores(Y,X);  
  jueces(X,Y): imparcial(X), imparcial(Y), not jueces (Y,X)}  
:- persona(X), persona(Y), X != Y.
```

3.1 Reglas de cardinalidad

Las reglas de cardinalidad son un subconjunto de las reglas de elección.

$$L \{a_1 : l_{11}, \dots, l_{1k}; \dots; a_n : l_{n1}, \dots, l_{nm}\} U$$

a_i : átomo del tipo `nombre(término*)`, donde `término` = constante o variable

l_{ij} : átomos del tipo `nombre(término*)`, donde `término` = constante o variables

$a_1, \dots, a_n$: átomos candidatos de elección

$l_{11}, \dots, l_{1k}; \dots; l_{n1}, \dots, l_{nm}$: condiciones locales para cada elección

L: cota inferior (número mínimo de átomos a_i ground, cuyas condiciones $l_{i1}, \dots$ se satisfacen, que deben ser verdaderos).

U: cota superior (máximo número de átomos a_i ground, cuyas condiciones $l_{i1}, \dots$ se satisfacen, que pueden ser verdaderos).

```
estudiante(eva).
```

```
2 {asignatura(eva,rcr); asignatura(eva,apa); asignatura(eva,eda)} 2.
```

Cualquier modelo estable contendrá exactamente dos asignaturas para eva.

- `asignatura(eva,apa) asignatura(eva,rcr) estudiante(eva)`
- `asignatura(eva,eda) asignatura(eva,rcr) estudiante(eva)`
- `asignatura(eva,eda) asignatura(eva,apa) estudiante(eva)`

```
estudiante(eva).  
estudiante(juan).  
estudiante(maria).  
nota_alta(eva).  
nota_alta(juan).
```

```
1 {becado(X): estudiante(X), nota_alta(X)} 3.
```

Tras el grounding: 1 {becado(eva), becado(juan)} 3.

Modelos estables resultantes:

- estudiante(eva) estudiante(juan) estudiante(maria) nota_alta(eva)
nota_alta(juan) becado(juan)
- estudiante(eva) estudiante(juan) estudiante(maria) nota_alta(eva)
nota_alta(juan) becado(eva)
- estudiante(eva) estudiante(juan) estudiante(maria) nota_alta(eva)
nota_alta(juan) becado(eva) becado(juan)

```
estudiante(eva).
estudiante(juan).
estudiante(maria).
primero(eva).
segundo(juan).
segundo(maria).

2{clase_manyana(X): estudiante(X), primero(X);
   clase_tarde(X): estudiante(X), segundo(X)} 2.
```

Tras el grounding:

```
2 {clase_manyana(eva), clase_tarde(juan), clase_tarde(maria)} 2.
```

Modelos estables resultantes:

- estudiante(eva) estudiante(juan) estudiante(maria) primero(eva) clase_manyana(eva) segundo(juan) segundo(maria) clase_tarde(maria)
- estudiante(eva) estudiante(juan) estudiante(maria) primero(eva) clase_manyana(eva) segundo(juan) segundo(maria) clase_tarde(juan)
- estudiante(eva) estudiante(juan) estudiante(maria) primero(eva) segundo(juan) segundo(maria) clase_tarde(juan) clase_tarde(maria)

valor(1..4). $\longrightarrow$ valor(1). valor(2). valor(3). valor(4).
columna(1..4).
fila(1..4).
sudoku(1,1,4).
sudoku(3,1,3).
sudoku(4,1,1).
sudoku(1,2,3).
sudoku(2,3,4).
sudoku(4,3,3).
sudoku(1,4,2).
sudoku(3,4,1).

1 {sudoku(X,Y,V): valor(V)} 1 :- columna(X), fila(Y).

2		1	
	4		3
3			
4		3	1

3. FUNCIONES AGREGADAS

`#count{X1, ..., Xk: l1, ..., ln}`

`Xi`: variable

`li`: condición/átomo del tipo nombre(término*), donde término = constante o variable

```
estudiante(eva).  
estudiante(juan).  
estudiante(sara).  
nota_alta(eva).  
nota_alta(sara).
```

```
total_excelentes(C) :- C = #count{X: estudiante(X), nota_alta(X)}.
```

```
excelencia :- #count{X: estudiante(X), nota_alta(X)} >= 2.
```

- estudiante(eva) estudiante(juan) estudiante(sara) nota_alta(eva) nota_alta(sara)
total_excelentes(2) excelencia

```
estudiante(eva).
estudiante(juan).
estudiante(ana).
asignatura(rcr).
asignatura(apa).
asignatura(eda).
matriculado(eva, rcr).
matriculado(juan, rcr).
matriculado(juan, apa).
matriculado(ana, eda).
```

% Una asignatura tiene alta demanda si hay al menos 2 estudiantes matriculados

```
alta_demanda(A) :- asignatura(A), #count {E, A : matriculado(E, A) } >= 2.
```

- ```
asignatura(rcr) asignatura(apa) asignatura(eda) matriculado(eva,rcr)
matriculado(juan,rcr) matriculado(juan,apa) matriculado(ana,eda)
alta_demanda(rcr) estudiante(eva) estudiante(juan) estudiante(ana)
```

$\#sum\{X_1, \dots, X_k: l_1, \dots, l_n\}$

$X_i$ : variable

$l_i$ : condición/átomo del tipo `nombre(término*)`, donde `término` = constante o variable

El resultado de  $\#sum \{ \dots \}$  es la **suma de los valores numéricos** asociados a cada tupla que cumple  $l_1, \dots, l_n$ . La primera variable de sum,  $X_1$ , debe representar el valor que se suma, y las demás sirven para distinguir tuplas.

```
producto(p1).
producto(p2).
producto(p3).
```

```
coste(p1,10).
coste(p2,20).
coste(p3,15).
```

```
elegido(p1).
elegido(p3).
```

*% Calcular el coste total de los productos elegidos*

```
coste_total(S) :- S = #sum {C,P : elegido(P), coste(P,C) }.
```

- |                 |              |              |              |              |
|-----------------|--------------|--------------|--------------|--------------|
| elegido(p1)     | elegido(p3)  | coste(p1,10) | coste(p2,20) | coste(p3,15) |
| coste_total(25) | producto(p1) | producto(p2) | producto(p3) |              |

## 4. OTRAS UTILIDADES

En clingo/ASP, el guion bajo `_` es la **variable anónima**:

- significa “aquí hay una variable, pero **no me importa su valor**”.
- se usa cuando necesitas que el átomo tenga ese argumento para que la regla esté bien formada, pero no vas a usar ese valor en el resto de la regla.

```
alumno(a1).
alumno(a2).
alumno(a3).
```

```
matricula(a1, logica, 2024).
matricula(a2, logica, 2024).
matricula(a3, ia, 2024).
```

```
% Hay una asignatura con alta demanda si tiene al menos 2 alumnos
matriculados
```

```
alta_demanda(A) :- asignatura(A), #count {X: matricula(X, A, _)} >= 2.
```